

UNIVERSIDAD RAFAEL LANDÍVAR

ÁREA DE INGENIERÍA

Programación Web



Actividad

Códigos de Estado HTTP

AUTOR

Sajvin Gómez, Mario Daniel (1612921)

Filtrar Producto

```
// Filtrar Productos
const productos = [
  { nombre: "Laptop", precio: 800, categoria: "Tecnología" },
  { nombre: "Mouse", precio: 20, categoria: "Tecnología" },
  { nombre: "Cafetera", precio: 50, categoria: "Electrodomésticos" },
  { nombre: "Silla", precio: 100, categoria: "Muebles" },
  { nombre: "Teléfono", precio: 500, categoria: "Tecnología" }
];

app.get('/productos', (req, res) => {
  let {categoria, precioMax} = req.query;
  let productosFiltrados = productos;

  // filtrar por categoria cuando se especifique
  if (categoria) {
    productosFiltrados = productosFiltrados.filter(
      (p) => p.categoria.toLowerCase() ===
categoria.toLocaleLowerCase()
    );
  }

  // filtrar por precio Maximo
  if (precioMax) {
    productosFiltrados = productosFiltrados.filter(
      (p) => p.precio <= parseFloat(precioMax)
    );
  }

  res.status(200).json(productosFiltrados);
})
```

The screenshot shows a REST client interface with the following details:

- Request:** GET `http://localhost:3000/productos/?categoria=Tecnologia`
- Status:** 200 OK, Size: 179 Bytes, Time: 14 ms
- Response:** A JSON array of 3 objects:

```
[
  {
    "nombre": "Laptop",
    "precio": 800,
    "categoria": "Tecnología"
  },
  {
    "nombre": "Mouse",
    "precio": 20,
    "categoria": "Tecnología"
  },
  {
    "nombre": "Teléfono",
    "precio": 500,
    "categoria": "Tecnología"
  }
]
```

The screenshot shows a web browser's developer tools with a new request tab. The request is a GET to `http://localhost:3000/productos/?precioMax=100`. The response is a 200 OK status with 179 bytes and a time of 5 ms. The response body is a JSON array of three objects:

```
[
  {
    "nombre": "Mouse",
    "precio": 20,
    "categoria": "Tecnología"
  },
  {
    "nombre": "Cafetera",
    "precio": 50,
    "categoria": "Electrodomésticos"
  },
  {
    "nombre": "Silla",
    "precio": 100,
    "categoria": "Muebles"
  }
]
```

Creación de Usuario

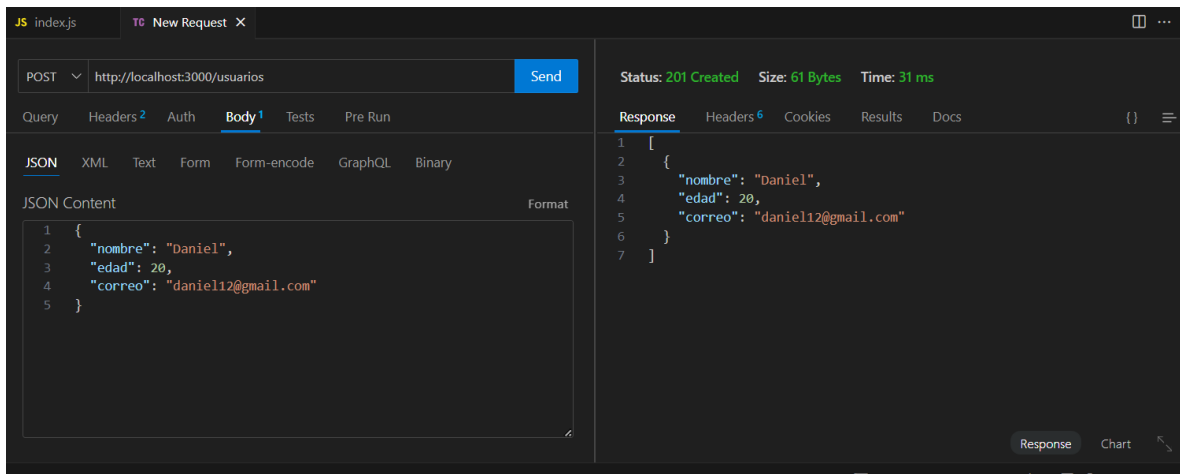
```
// Creación de usuarios

const usuarios = []

app.post('/usuarios', (req, res) => {
  const {nombre, edad, correo} = req.body;

  // validaciones
  if (!nombre || !edad || !correo) {
    res.status(400).json({ error: "Todos los campos (nombre, edad, correo) son obligatorios" });
  }
  if (typeof nombre !== "string" || typeof correo !== "string") {
    return res.status(400).json({ error: "Nombre y correo deben ser cadenas de texto" });
  }
  if (typeof edad !== "number" || edad <= 0) {
    return res.status(400).json({ error: "La edad debe ser un numero." });
  }

  // crear el usuario
  const nuevoUsuario = {nombre, edad, correo};
  usuarios.push(nuevoUsuario);
  res.status(201).json(usuarios);
});
```



Actualizar datos de un usuario

```
// actualizar datos de un usuario
const usuarios = [
  { id: 1, nombre: "Juan Pérez", edad: 25 },
  { id: 2, nombre: "Ana López", edad: 30 },
  { id: 3, nombre: "Carlos Ramírez", edad: 22 }
];

// Endpoint PUT /usuarios/:id (Actualizar usuario)
app.put("/usuarios/:id", (req, res) => {
  const id = parseInt(req.params.id);
  const { nombre, edad } = req.body;

  // Buscar el usuario por ID
  const usuario = usuarios.find((u) => u.id === id);

  if (!usuario) {
    return res.status(404).json({ error: "Usuario no encontrado" });
  }

  // Validaciones opcionales antes de actualizar
  if (nombre && typeof nombre !== "string") {
    return res.status(400).json({ error: "El nombre debe ser una cadena de texto" });
  }
  if (edad && (typeof edad !== "number" || edad <= 0)) {
    return res.status(400).json({ error: "La edad debe ser un número positivo" });
  }
});
```

```
// Actualizar solo los campos proporcionados
if (nombre) usuario.nombre = nombre;
if (edad) usuario.edad = edad;

res.json({ mensaje: "Usuario actualizado exitosamente", usuario });
});
```

The screenshot displays a REST client interface with a PUT request to `http://localhost:3000/usuarios/3` and its corresponding JSON response.

Request Details:

- Method: PUT
- URL: `http://localhost:3000/usuarios/3`
- Body (JSON):

```
{
  "id": 5,
  "nombre": "Angel",
  "edad": 3
}
```

Response Details:

- Status: 200 OK
- Size: 91 Bytes
- Time: 8 ms
- Body (JSON):

```
{
  "mensaje": "Usuario actualizado exitosamente",
  "usuario": {
    "id": 3,
    "nombre": "Angel",
    "edad": 3
  }
}
```

Eliminar Usuario

```
// Eliminar usuario
app.delete("/usuarios/:id", (req, res) => {
  const id = parseInt(req.params.id);

  // Buscar el índice del usuario
  const indice = usuarios.findIndex((u) => u.id === id);

  if (indice === -1) {
    return res.status(404).json({ error: "Usuario no encontrado" });
  }

  // Eliminar usuario del arreglo
  usuarios.splice(indice, 1);

  res.json({ message: "Usuario eliminado correctamente" });
});
```

